Lecture 9

UML Class Diagrams

Software design

- **Design**: specifying the structure of how a software system will be written and function, without actually writing the programs/implementation yet
- A transition from "what" the system must do, to "how" the system will do it
- Design is what we need to implement a system that meets our requirements
- What fields and methods will each class have?
- How will the classes interact with each other?

Describing designs with UML diagrams

- Class diagram
  - Shows classes and relationships among them.
  - Shows fields and methods, but not anything what interacts but not what happens when they do interact.
- Sequence diagram
  - Shows how objects communicate.
  - Shows the order of the interaction, describing how objects collaborate: what messages a receiver and when.
- Activity diagram
  - Describes how the function work, the algorithm of the function.

What is UML?

- Pictures or views of an OO system
  - Programming languages are not abstract enough for OO design
  - UML is not a process, but lots of companies use it
- What is legal UML?
  - A descriptive language: shaped by usage and convention
  - Vague and informal (UML means what people mean by modeling)
  - It's okay to omit things from UML diagrams if they aren't needed by team/supervisor/instructor

UML: Unified Modeling Language

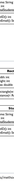
- Union of Many Languages
  - Class diagrams
  - Use diagrams
  - Activity diagrams
  - Sequence diagrams
  - Collaboration diagrams
  - State/Start diagrams
  - Component diagrams
  - Deployment diagrams
  - ...

Uses for UML

- As a **sketch**: to communicate aspects of system
  - Forward engineering (UML before coding)
  - Reverse engineering (UML after coding, as documentation
  - Often done on whiteboard on paper
  - Leads to rough sketches of ideas
- As a **blueprint**: detailed design to be implemented
  - Sometimes even with CASE (computer-aided software engineering) tools
- As a **programming language**: with the right tools, code can be auto-generated and even edited
  - Only good if the is faster than coding in a "real" language

Class Diagram

- Class name
  - Extra stereotypes on top of the basic name.
  - Can be italics for an abstract class name
- Attributes (optional)
  - Kinds of the class
- Operations/methods (optional)
  - Long and (mid-)grained verbs.
  - but don't omit any methods from an interface!
  - should not include inherited methods



Attributes

- `visibility name : type`
- **visibility**
  - + public
  - # protected
  - - private
  - ~ package (default)
  - / derived
- **derived attributes**: not stored, but can be computed from other attributes
- Underline static attributes

Operations

- `visibility name(parameters) : return_type`
- **visibility**
  - + public
  - # protected
  - - private
  - ~ package (default)
- **parameters** listed as **name : type**
- static methods are underlined
- **constructor type**: on constructors and when return type is void

Comments

- Represented as a folded note, attached to the appropriate class/method/etc by a dashed line

Relationships between classes

- **Generalization**: or inheritance relationship
  - Inheritance between classes
  - Interface implementation
- **Association**: a usage relationship
  - Aggregation
  - Composition

Generalization relationships

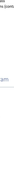
- Represents class hierarchy
- Arrows point from child to parent
- Class arrow styles indicate if parent is abs:
  - **Line**: solid line, white arrow
  - **Interface**: dashed line, white arrow
- Often omit trivial / obvious generalization relationships, such as drawing the Object class as a parent

Association types

- **Aggregation**
  - Drawing: "is part of"
  - + provided but is less white diamond
  - Implementation:
  - + composing: "is entirely made of" the stronger version of aggregation
  - + Parent: child and the side this whole
  - + specified by a black diamond
- **Dependency**
  - Drawing: "uses temporarily"
  - + specified by dashed line
  - + often is an implementation detail, not essentially part of the object's state

Associational (usage) relationships

1. Multiplicity (how many are used)
   - 1 (one and only one)
   - 1+ (one or more)
   - 0..1 (optional, 0 or 1 instance)
   - 5..7 (5 or more, 7 may be omitted)
2. Role name
   - name of relationship in the class's field
3. Navigability (direction)
   - In the example, A uses B



Association multiplicities

- **One-to-one**
  - Each car has exactly one engine.
  - Each engine belongs to exactly one car
- **One-to-many**
  - Each book has many pages.
  - Each page belongs to exactly one book.

Aggregation / composition example

- If the cinema goes away
  - the director lies behind other composition
  - but movies may still exist: aggregation

Class diagram example: people

Tools for creating UML diagrams

- Diagrams.net (draw.io)
- LucidChart
- StarUML
- Visual Paradigm (online/offline)

Class diagrams are great for:

- discovering related data and attributes
- getting a quick picture of the important entities in a system
- seeing whether you have too few/many classes
- seeing whether the relationships between objects are too complicated, too simple, too many one-to-many, etc
- ordering dependencies between one class/object and another

Class diagrams are not so great for:

- discovering algorithms (not data-driven) behavior
- Finding the flow of steps for objects to solve a given problem (understanding the app's overall control flow (event-driven? web-based? sequential? etc.)

UML class diagram: Summary

- UML is a language for describing various aspects of software designs.
- UML class diagrams present a static view of the system, displaying classes and relationships between them.

How to create class diagram?

- Identify classes and interactions from project requirements:
  - **Nouns** are potential classes, objects, and fields
  - **Verbs** are potential methods (should be classes?
    - Which nouns are fields?
  - **Verbs** are potential methods or responsibilities of a class
  - **Verbs** describing interactions are potential interactions (containment, generalization, dependence, etc.)

How to create an analysis class diagram

- Map entities to domain classes
- Map relationships to associations
  - cardinality constraints to class cardinalities
- Map associative entities to association classes
- Write constraint statements (if any)